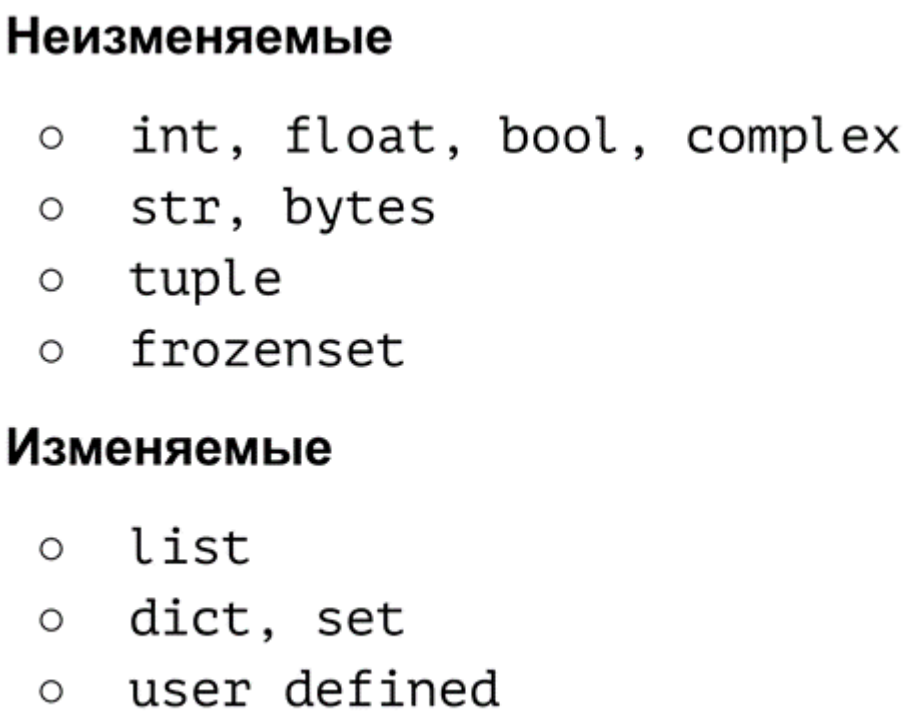
python.ipynb

Инфо по python:

- https://education.yandex.ru/handbook/python (https://education.yandex.ru/handbook/python)
- https://pythonworld.ru/ (https://pythonworld.ru/)
- https://docs.python.org/3.13/ (https://docs.python.org/3.13/)

Типы данных

Переменные делятся на 2 группы



Неизменяемые:

```
int - целые числа # пример: 0, 2, 5
float - числа с плавающей точкой (дробные) # пример: 0.0, 2.4
bool - булевые (0, 1 или True, False) # пример: True, False
complex - комплексные числа # пример: 1+0j, 2+2j
str - строковые # пример: 'Привет', "sad"
bytes - байтовые строки # пример: b'\xd0\x9f\xd1\x80\xd0\xb8\xd0\xb2\xd0\xb5\xd1\x82' - это Привет
tuple - набор фиксированной длины # пример: (0,1), (True, 1, 'a12')
frozenset - множество # пример: frozenset('qwerty')
```

Числа int, float, complex

Целые числа (int)

```
num = 42
num = 42_000_000
num = 0o10  # 8
num = 0x10  # 16
```

Вещественные числа (float)

```
num = 3.14
```

Комплексные числа (complex)

```
num = 9 + 0.75j  # num.real, num.imag
```

Арифметические операции

- Сложение: +
- Вычитание: -
- Умножение: *
- Деление: /
- Целочисленное деление: //
- Остаток от деления: %
- Возведение в степень: **

Битовые операции

- Умножение: &
- Сложение: |
- Инверсия: ~
- Исключающее ИЛИ: ^
- Побитовый сдвиг: <<, >>

Если выполнять операции int с float, то итоговое число будет float

```
num_a = 42     # int
num_b = 3.0    # float
num_a /num_b   # float
```

```
num_a = 42     # int
num_b = 3.0    # float
num_a /num_b   # float
```

```
a = 4
b = 3
a * b
```

Также операцию возведение в степень можно заменить на встроенную функцию **pow**

pow(число, степень)

Пример:

```
num = 12
pow(num, 2) # 144
num ** 2 # 144
```

```
# code
12 ** 2
pow(12, 2)
```

bool

Логический тип (bool)

```
is_true = True  # или False
```

- Логическое И: **and**
- Логическое ИЛИ: **or**
- Отрицание: **not**

Вычисление составных логических выражений **ленивое:**

```
return_false() and return_true()

return_true() or return_false()
```

Операции сравнения:

1. **==** — равно
2. **!=** — не равно
3. **>** — больше
4. **<** — меньше
5. **>=** — больше или равно
6. **<=** — меньше или равно Пример:

```
a = 5
b = 5
c = 10
a == b
a != c
b <= c
```

```
a = 5
a == 0
```

[Булева алгебра (https://ru.wikipedia.org/wiki/%D0%91%D1%83%D0%BB%D0%B5%D0%B2%D0%B0_%D0%B0%D0%BB%D0%B3%D0%B5%D0%B1%D1%80%D0%B0)](https://ru.wikipedia.org/wiki/%D0%91%D1%83%D0%BB%D0%B5%D0%B2%D0%B0_%D0%B0%D0%BB%D0%B3%D0%B5%D0%B1%D1%80%D0%B0)

```
True = 1
False = 0
```

1. Логическое **И** может быть задано оператором: **&** или **and**

```
True & False # False
True and False # False
```

2. Логическое **Или** может быть задано оператором: **|** или **or**

```
True | False # True
True or False # True
```

3. Логическое **Отрицание** может быть задано оператором: **~** или **not**

Python оператор ~ является побитовым оператором NOT. Он инвертирует все биты числа.

```
~True # -2 Это происходит потому, что ~True эквивалентно -(x + 1). Таким образом, ~1 становится -(1 + 1), что равно -2.
not(True) # False
```

Операции сравнения дают нам булево значения.

Пример:

```
a = 5
b = 10
a > b # False
a < b # True
a == b # False
a != b # True
```

```
a = 5
not(a < 0)
```

```
a = 5
a != 9
```

Строки (str, bytes)

```
s = "просто строка"  # str
byte_s = b"qwerty"  # bytes
raw_s = r"111\nданные"  # str

str.encode() -> bytes
bytes.decode() -> str

str1 + str2  # конкатенация
"a" * 5  # повторение "aaaaa"
```

Форматирование

- "x %s %d" % ("y", 42)
- "x %(y)s %(n)d" % dict(y="y", n=42)
- "x {} {}".format("y", 42)
- y, n = "y", 42

  f"x {y} {n}"

"x y 42"

Строки можно складывать

```
str1 = 'Hello '
str2 = 'Bon jour'
str1 + str2 # Hello Bon jour
```

```
# code
str1 = 'Hello'
str2 = ' Bio'
str1 + str2 # Hello Bio
```

Строки умножать на число (строка повторится n раз)

```
3 * 'Hello' # HelloHelloHello
```

```
# code
5 * ' Hello'
```

Встроенные методы и функции

S - строка

1. Длина строки

```
len('bio') # 3
```

2. Доступ по индексу

```
word = 'bio'
word[0] # b
```

3. Извлечение среза

```
word = 'biotech'
word[2:4] # ot
```

4. Поиск подстроки в строке. Возвращает номер первого вхождения или -1: **S.find(str, [start],[end])**

```
word = 'biobio'
word.find('bio') # 0
```

5. Поиск подстроки в строке. Возвращает номер последнего вхождения или -1: **S.rfind(str, [start], [end])**

```
word = 'biobio'
word.rfind('bio') # 3
```

6. Замена шаблона на замену. maxcount ограничивает количество замен: **S.replace(шаблон, замена[, maxcount])**

```
word = 'biotech'
word.replace('bio', 'HSE') # HSEtech
```

7. Разбиение строки по разделителю: **S.split(символ)**

```
word = 'HSe bio tech'
word.split(' ') # ['HSe', 'bio', 'tech']
```

8. Состоит ли строка из цифр: **S.isdigit()**

```
word = '1234'
word.isdigit() # True
```

9. Состоит ли строка из букв: **S.isalpha()**

```
word = 'asd'
word.isalpha() # True
```

10. Состоит ли строка из цифр или букв: **S.isalnum()**

```
word = 'asd123'
word.isalnum() # True
```

11. Преобразование строки к верхнему регистру: **S.upper()**

```
word = 'HsEbIo'
word.upper() # HSEBIO
```

12. Преобразование строки к нижнему регистру: **S.lower()**

```
word = 'HsEbIo'
word.upper() # hsebio
```

13. Состоит ли строка из символов в нижнем регистре: **S.islower()**

```
word = 'HsEbIo'
word.islower() # False
```

14. Состоит ли строка из символов в верхнем регистре: **S.isupper()**

```
word = 'HsEbIo'
word.isupper() # False
```

И все остальные функции и методы: [https://pythonworld.ru/tipy-dannyx-v-python/stroki-funkcii-i-metody-strok.html (https://pythonworld.ru/tipy-dannyx-v-python/stroki-funkcii-i-metody-strok.html)](https://pythonworld.ru/tipy-dannyx-v-python/stroki-funkcii-i-metody-strok.html)

```
# code
word = 'HsEbIo'
word.lower() # HSEBIO
```

```
[start:finish:step]
```

Получить байтову строку
`'Байты'.encode('utf-8')`

```
# code
'Привет'.encode('utf-8')
```

tuple

tuple - (кортеж), по сути - неизменяемый список (list)

tuple можно задать, **tuple** или **()** Пример:

```
t1 = (1, 3, 'asd') # (1, 3, 'asd')
t2 = tuple('asd') # автоматический бьет строку на символы ('a', 's', 'd')
t3 = tuple(['asd']) # ('asd', )
```

```
# code
tuple(['asd'])
```

```
a = 5
b = 6
c = 9
(123, 'hse', 'bio')
```

```
tuple('asd')
```

Изменяемые:

```
list – лист (массив, только элементом может быть, что угодно) # пример: [0, True, 'hello', [1,2]]
dict – словарь # пример: {key: value}
set – множество # пример: {0, 12}
user defined – пользовательский тип данных
```

list

list - упорядоченные изменяемые коллекции объектов произвольных типов (почти как массив, но типы могут отличаться).

Если коротко, то list - это массив, тоолько с поддержкой разных типов.

list можно задать, **list** или **[ ]**

Пример:

```
l1 = [1, 'qwe', [1,0], (1.4,5)] # [1, 'qwe', [1, 0], (1.4, 5)]
l2 = list([1, 'qwe', [1,0], (1.4,5)]) # [1, 'qwe', [1, 0], (1.4, 5)]
```

```
l1 = [1, 'qwe', [1,0], (1.4,5)]
l2 = list([1, 'qwe', [1,0], (1.4,5)])
```

Список (list) - можно изменять!

1. По индексу получать (изменять) элемент

```
l1 = ['один', 'два', 'три', 'четыре', 'пять']
l1[1] = '2'
```

2. Делать срез (получать и изменять)

```
l1 = ['один', 'два', 'три', 'четыре', 'пять']
l1[1:3] # ['два', 'три']
```

3. Получать размер списка с помощью функции: **len** (как у строк)

```
l1 = ['один', 'два', 'три', 'четыре', 'пять']
len(l1) # 5
```

```
# code

l1 = ['один', 'два', 'три', 'четыре', 'пять']
l1[1]
```

Методы списков:

1. Добавляет элемент в конец списка: **list.append(x)**

```
l1 = []
l1.append('bio') # ['bio']
```

2. Расширяет список list1, добавляя в конец все элементы списка list2: **list1.extend(list2)**,

```
l1 = ['bio']
l2 = ['HSE']
l1.extend(l2) # ['bio', 'HSE']
```

можно заменить сложением списков list1+list2 (создается 3 список)

```
l1 = ['bio']
l2 = ['HSE']
```

```
l1 + l2 # ['bio', 'HSE']
l1 # ['bio']
```

3. Вставляет на i-ый элемент значение x: **list.insert(i, x)**

```
l1 = ['bio', 'HSE', 'asd']
l1.insert(1, '12') # ['bio', '12', 'HSE', 'asd']
```

4. Удаляет первый элемент в списке, имеющий значение x: **list.remove(x)**

```
l1 = ['bio', 'HSE', 'asd']
l1.insert(1, '12') # ['bio', '12', 'HSE', 'asd']
```

5. Удаляет i-ый элемент и возвращает его. Если индекс не указан, удаляется последний элемент: **list.pop([i])**

```
l1 = ['bio', 'HSE', 'asd']
del_elem = l1.pop(0) # 'bio'
# l1 ['HSE', 'asd']
l1.pop()
l1 ['HSE']
```

6. Возвращает количество элементов со значением x: **list.count(x)**

```
l1 = ['bio', 'HSE', 'bio']
l1.count('bio') # 2
```

7. Сортирует список на основе функции (без функции будет сортировка по возрастанию): **list.sort([key=функция])**

```
l1 = [3, 1, 2]
l1.sort() # [1, 2, 3]
```

8. Разворачивает список: **list.reverse()**

```
l1 = [3, 1, 2]
l1.reverse() # [2, 1, 3]
```

9. Очищает список: **list.clear()**

```
l1 = [3, 1, 2]
l1.clear() # []
```

И все остальные методы: [https://pythonworld.ru/tipy-dannyx-v-python/spiski-list-funkcii-i-metody-spiskov.html (https://pythonworld.ru/tipy-dannyx-v-python/spiski-list-funkcii-i-metody-spiskov.html)](https://pythonworld.ru/tipy-dannyx-v-python/spiski-list-funkcii-i-metody-spiskov.html)

```
# code
l1 = ['123', '213324', 'asd']
l1.sort(key=len)
```

```
l1
```

dict

Словарь (dict) - неупорядоченные коллекции произвольных объектов с доступом по ключу.

dict можно задать, **dict** или **{}**

```
d1 = {
  'Python КиМБ232': 1,
  'Python КиМБ231': 2,
  'Анализ данных ВШЭ БКОНЕ 2й курс': 4
  }
```

```
d2 = dict(
  [
  ('Python КиМБ232', 1),
  ('Python КиМБ231', 2),
  ('Анализ данных ВШЭ БКОНЕ 2й курс', 4)
  ]
)
```

1. Можно получать доступ к значению через ключ

```
d = {
  'Python КиМБ232': 1,
  'Python КиМБ231': 2,
  'Анализ данных ВШЭ БКОНЕ 2й курс': 4
  }

d['Python КиМБ232'] # 1
```

2. Можно получать размер словарая через **len**

```
d = {
  'Python КиМБ232': 1,
  'Python КиМБ231': 2,
  'Анализ данных ВШЭ БКОНЕ 2й курс': 4
  }

len(d) # 3
```

```
# code
{231: []}[231]
```

Методы

1. Очищает словарь: **dict.clear()**

```
d = {'key': 1}
d.clear() # {}
```

2. Возвращает значение ключа: **dict.get(key[, default])**, можно **обращатся по ключу d[key]**

```
d = {
  'Python КиМБ232': 1,
  'Python КиМБ231': 2,
  'Анализ данных ВШЭ БКОНЕ 2й курс': 4
  }
d.get('Python КиМБ232') # 1
d['Python КиМБ232'] # 1
```

3. Возвращение пары (ключ, значение): **dict.items()**

```
d = {
  'Python КиМБ232': 1,
  'Python КиМБ231': 2,
  'Анализ данных ВШЭ БКОНЕ 2й курс': 4
  }
d.items() # dict_items([('Python КиМБ232', 1), ('Python КиМБ231', 2), ('Анализ данных ВШЭ БКОНЕ 2й
курс', 4)])
list(d.items())  # list с tuple (ключ, значение)
```

4. Получение ключей: **dict.keys()**

```
d = {
  'Python КиМБ232': 1,
  'Python КиМБ231': 2,
```

```
  'Анализ данных ВШЭ БКОНЕ 2й курс': 4
  }
d.keys() #dict_keys(['Python КиМБ232', 'Python КиМБ231', 'Анализ данных ВШЭ БКОНЕ 2й курс'])
list(d.keys())  # list с ключами
```

5. Удаляет ключ и возвращает значение: **dict.pop(key[, default])**

```
d = {
  'Python КиМБ232': 1,
  'Python КиМБ231': 2,
  'Анализ данных ВШЭ БКОНЕ 2й курс': 4
  }
d.pop('Python КиМБ231') # {'Python КиМБ232': 1, 'Анализ данных ВШЭ БКОНЕ 2й курс': 4}
```

6. Удаляет с конца LIFO и возвращает пару (ключ, значение): **dict.popitem()**

```
d = {
  'Python КиМБ232': 1,
  'Python КиМБ231': 2,
  'Анализ данных ВШЭ БКОНЕ 2й курс': 4
  }
d.popitem() #{'Python КиМБ232': 1, 'Python КиМБ231': 2}
```

7. Обновляет словарь, добавляя пары (ключ, значение) из other: **dict.update([other])**

```
d = {
  'Python КиМБ232': 1,
  }
d.update([('Python КиМБ231',2)]) #{'Python КиМБ232': 1, 'Python КиМБ231': 2}
```

Можно еще так по ключу: **dict[key] = values**

```
d = {
  'Python КиМБ232': 1,
  }
d['Python КиМБ231']= 2 #{'Python КиМБ232': 1, 'Python КиМБ231': 2}
```

8. Возвращает значения в словаре: **dict.values()**

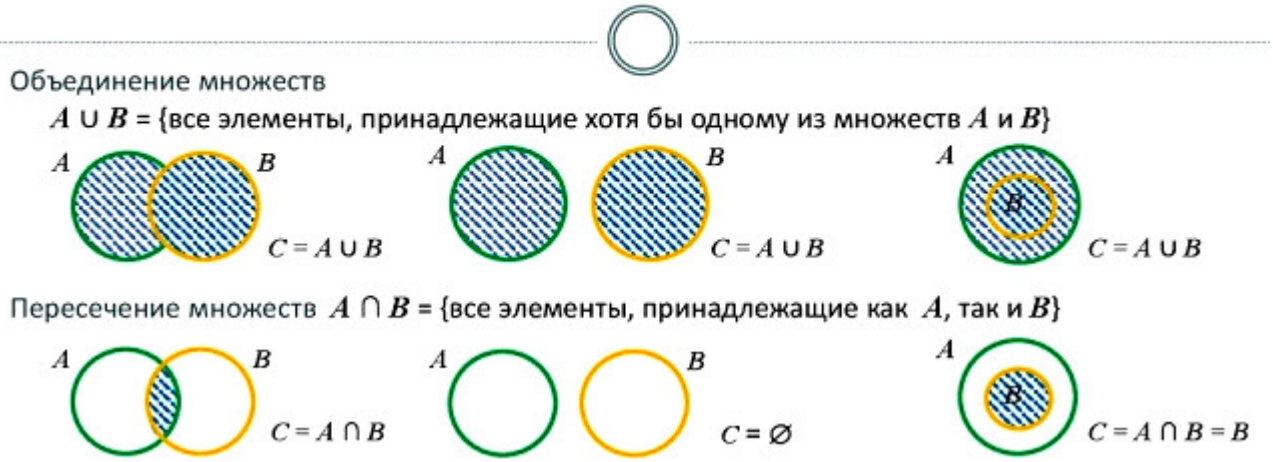
```
d = {
'Python КиМБ232': 1,
'Python КиМБ231': 2,
'Анализ данных ВШЭ БКОНЕ 2й курс': 4
}
d.values() # dict_values([1, 2, 4])
list(d.values()) # [1, 2, 4]
```

И все остальные методы: [https://pythonworld.ru/tipy-dannyx-v-python/slovari-dict-funkcii-i-metody-slovarej.html (https://pythonworld.ru/tipy-dannyx-v-python/slovari-dict-funkcii-i-metody-slovarej.html)](https://pythonworld.ru/tipy-dannyx-v-python/slovari-dict-funkcii-i-metody-slovarej.html)

set

Множество (set) - "контейнер", содержащий **не повторяющиеся элементы** в случайном порядке

Операции над множествами



Можно задать: **{ }** или **set( )**

```
s1 = set((345, 14, 234, 0)) # {0, 14, 234, 345}
s2 = {345, 14, 234, 0} # {0, 14, 234, 345}
```

1. Объединение нескольких множеств: **set.union(other, ...) или set | other | ...**

```
d1 = {1,2,3}
d2 = {3,4,5}
d1.union(d2) # {1, 2, 3, 4, 5}
```

2. Пересечение: **set.intersection(other, ...) или set & other & ...**

```
d1 = {1,2,3}
d2 = {3,4,5}
d1.intersection(d2) # {3}
```

3. Принадлежит ли x множеству s: **x in s** (аналогично list, dict)

```
d = {1,2,3}
1 in d # True
4 in d # False
```

4. Добавляет элемент в множество - **set.add(elem)**

```
d = {1,2,3}
d.add(4) # {1, 2, 3, 4}
```

5. Удаляет элемент из множества - **set.remove(elem)**

```
d = {1,2,3}
d.remove(1) # {2, 3}
```

6. Удаляет элемент, если он находится в множестве: **set.discard(elem)**

```
d = {1, 2, 3}
d.discard(4) # Если бы использовался метод remove, возникла бы ошибка
```

7. Удаляет первый элемент из множества: **set.pop()**

```
d = {1, 2, 3}
d.pop(d2) # {2, 3}
```

8. Очистка множества: **set.clear()**

```
d = {1,2,3}
d.clear() # { }
```

Вся доп. информация о множествев python: https://pythonworld.ru/tipy-dannyx-v-python/mnozhestva-set-i-frozenset.html (https://pythonworld.ru/tipy-dannyx-v-python/mnozhestva-set-i-frozenset.html)

```
# code
```

```

```

Ввод и вывод данных

Для вывода в консоль в питоне, существует встроенная функция под названием print()

Пример использования:

```
print('hello')

print('HSE', 'Bio')

print("Moscow")

print("""Всем
Привет
"""
  )
```

```
#print('Hello', 'Bio')
# print('hello')

# print('HSE', 'Bio')

# print("Moscow")

# print("""Всем
# Привет
# """
#   )
# print("""
# Всем
# Привет
# """)
```

Для ввода данных с клавиатуры, в Python используется функция input(), которая возвращает в программу введённую пользователем строку

P.S.Данные в программу могут поступать различными способами.

Пример использования:

```
phrase1 = input()
print(phrase1)

phrase2 = input('Введите что-нибудь: ')
print(phrase2)
```

```
phrase1 = input()
print(phrase1)
```

```
phrase2 = input('Введите что-нибудь: ')
print(phrase2)
```

```
phrase1 = input()
phrase1
```

По умолчанию input(): передает, в питон строковый (**str**) тип:
`type(input())`

```
type(input())
```

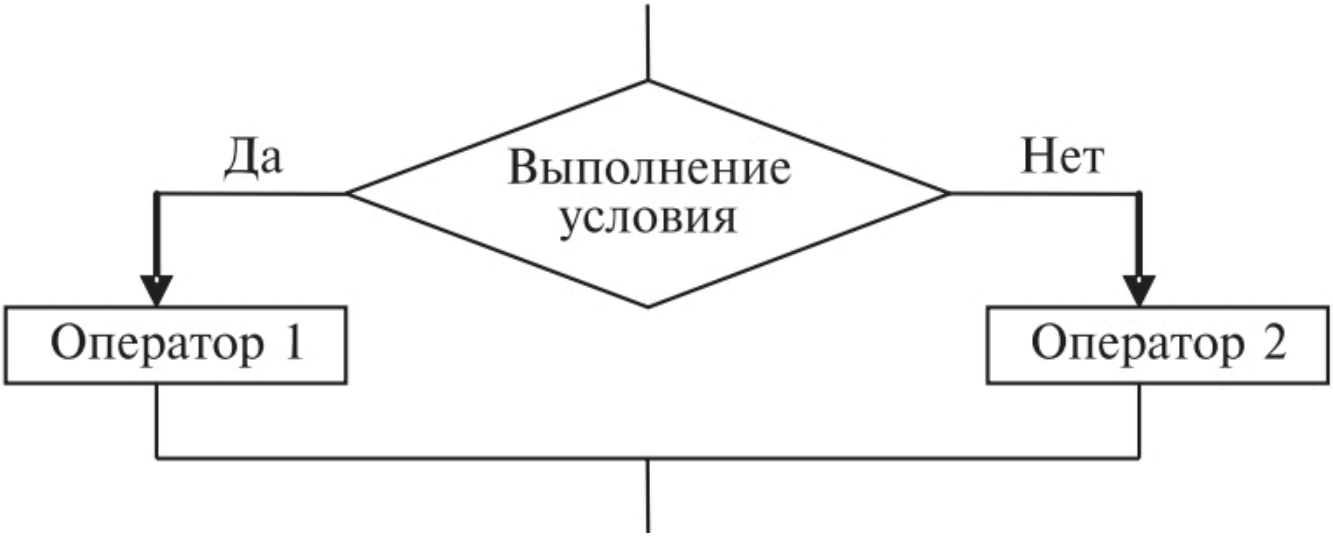
```
a = 123
b = 's'
c = 0.5
type(a)
```

```

```

Условные операторы, Циклы

Условные операторы



Оператор if является началом условной конструкции.

Тело условной конструкции может содержать одно или несколько выражений (строк). По завершении тела может идти следующее условие, которое начинается с оператора elif (сокращение от else if — «иначе если»). Оно проверяется только в случае, если предыдущее условие не было истинным.

Синтаксис в elif аналогичен if. Операторов elif для одного блока условного оператора может быть несколько, а может не быть совсем. Последним идёт оператор else, который не содержит условия, а выполняется, только если ни одно из предыдущих условий в if и elif не выполнилось. Оператор else не является обязательным.

```
a = 5
if a >= 0:
```

```
   print('a >= 0')
elif 4 <= a <= 5:
   print('4 <= a <= 5')
else:
   print('a < 0')
```

Условия можно писать в строку:

```
a = 5
print('a >= 0') if a >= 0 else print('a < 0')
```

Доп. информация: https://education.yandex.ru/handbook/python/article/uslovnyy-operator (https://education.yandex.ru/handbook/python/article/uslovnyy-operator)

```
a = 0
if a <= 0 and a>5:
  print(a)
elif a != 0:
  print(0)
```

Циклы

Схема цикла



Цикл for

Цикл **for** используется для итерации по элементам итерируемого объекта, такого как список, кортеж, строка или диапазон чисел.

1. Итерация по списку:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
     print(fruit) # выведет каждый элемент списка fruits
```

2. Итерация по диапазону чисел:

```
for i in range(5): # range — генератор
     print(i) # выведет числа от 0 до 4
```

```
 for i in range(5): # range — генератор
      print(i) # выведет числа от 0 до 4
```

Цикл while

Цикл **while** выполняет блок кода до тех пор, пока условие остается истинным.

1. Простой цикл while:

```
count = 0
while count < 5: # пока count < 5, цикл будет работать
     print(count)
     count += 1
```

```
 #code
 count = 0
 while count < 5: # пока count < 5, цикл будет работать
   print(count)
   count = count + 2 # count += 2
```

```
 # count = 0
 # while True:
 #   count = count + 1
 #   print(count)
```

Прерывание (break) и пропуск итерации цикла (continue)

break: Используется для немедленного завершения цикла, даже если условие еще истинно.

1.

```
for i in range(10):
      if i == 5:
          break
      print(i)
```

1.

```
count = 0
while count < 5:
      if count == 3:
          break
      print(i)
      count += 1
```

```
# code
for i in range(10):
   if i == 5:
       k = i
       break
   print(i)
print(k)
```

```
a = input()

while len(a) < 5:
  if a[:2].isdigit():
    continue

  print(a)
  a = input()
  if len(a) == 7:
    break

print(a)
```

continue: Используется для пропуска текущей итерации цикла и перехода к следующей

1.

```
for i in range(10):
      if i % 2 == 0:
          continue
      print(i) # вывод от 1 до 9 нечетных чисел
```

2.

```
count = 0
while count < 10:
      count += 1
      if count % 2 == 0:
          continue
      print(count)  # вывод от 1 до 9 нечетных чисел
```

```
# code
for i in range(10):
   if i % 2 == 0:
       continue
   print(i)
```

else после циклов

После цикла for или while можно использовать блок else, который будет выполнен только в том случае, если цикл закончился естественным образом, то есть без использования операторов break. Если в цикле был использован оператор break, то блок else будет проигнорирован.

1. Пример цикла for с else (блок else выполняется после завершения цикла for):

```
for num in range(5):
     print(num)
else:
     print("Числа закончились")
#Вывод
0
```

```
1
2
3
4
Числа закончились
```

2. Пример цикла for с else и break в цикле (из-за break блок else не выполняется):

```
for num in range(5):
     if num == 3:
         break
     else:
         print(num)
 else:
     print("Числа закончились")
#Вывод
0
1
2
```

continue не влияет на блок else

```
# code
for i in range(10): # [0, 8)
  if i % 2 == 0:
    print(i)
  if i == 15:
    break
else:
  print('Hello')
```

Вложенные циклы

Циклы могут быть вложенными друг в друга, что позволяет выполнять более сложные итерации. Если коротко, это цикл в цикле:

```
for i in range(1, 10):
    for j in range(1, 10):
        print(f"{i*j:4}", end=" ")
    print()
```

или

```
count1 = 0
while count1 <= 5:
  count2 = 0
  while count2 <=3:
    count2 += 1
  count1 += 1
```

Также можно:

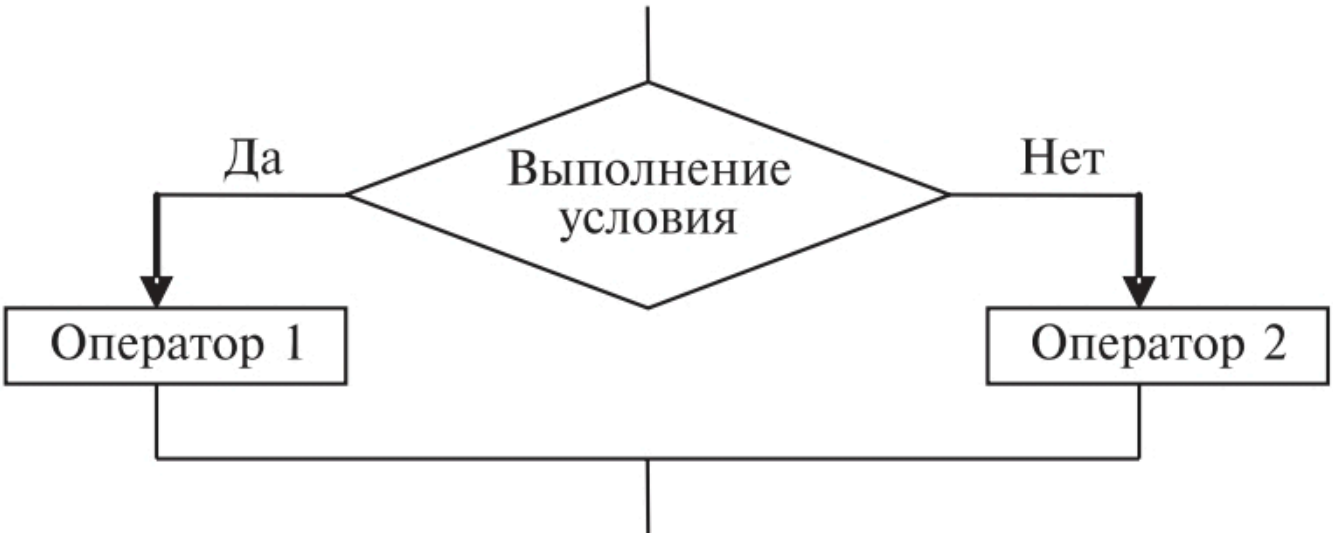
```
for i in range(10):
  while ...
```

или наоборот

```
for i in range(1, 10, 2):
  print(i)
```

Условные операторы, Циклы

Условные операторы



Оператор if является началом условной конструкции.

Тело условной конструкции может содержать одно или несколько выражений (строк). По завершении тела может идти следующее условие, которое начинается с оператора elif (сокращение от else if — «иначе если»). Оно проверяется только в случае, если предыдущее условие не было истинным.

Синтаксис в elif аналогичен if. Операторов elif для одного блока условного оператора может быть несколько, а может не быть совсем. Последним идёт оператор else, который не содержит условия, а выполняется, только если ни одно из предыдущих условий в if и elif не выполнилось. Оператор else не является обязательным.

```
a = 5
if a >= 0:
   print('a >= 0')
elif 4 <= a <= 5:
   print('4 <= a <= 5')
else:
   print('a < 0')
```

Условия можно писать в строку:

```
a = 5
print('a >= 0') if a >= 0 else print('a < 0')
```

Доп. информация: https://education.yandex.ru/handbook/python/article/uslovnyy-operator (https://education.yandex.ru/handbook/python/article/uslovnyy-operator)

```
a = 0
if a <= 0 and a>5:
  print(a)
elif a != 0:
  print(0)
```

Циклы

Схема цикла



Цикл for

Цикл **for** используется для итерации по элементам итерируемого объекта, такого как список, кортеж, строка или диапазон чисел.

1. Итерация по списку:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
     print(fruit) # выведет каждый элемент списка fruits
```

2. Итерация по диапазону чисел:

```
for i in range(5): # range — генератор
     print(i) # выведет числа от 0 до 4
```


```
for i in range(5): # range — генератор
     print(i) # выведет числа от 0 до 4
```


Цикл while

Цикл **while** выполняет блок кода до тех пор, пока условие остается истинным.

1. Простой цикл while:

```
count = 0
while count < 5: # пока count < 5, цикл будет работать
     print(count)
     count += 1
```

```
#code
count = 0
while count < 5: # пока count < 5, цикл будет работать
  print(count)
  count = count + 2 # count += 2
```

```
# count = 0
# while True:
#   count = count + 1
#   print(count)
```

Прерывание (break) и пропуск итерации цикла (continue)

break: Используется для немедленного завершения цикла, даже если условие еще истинно.

1.

```
for i in range(10):
      if i == 5:
          break
      print(i)
```

1.

```
count = 0
while count < 5:
      if count == 3:
          break
      print(i)
      count += 1
```

```
# code
for i in range(10):
   if i == 5:
       k = i
       break
   print(i)
print(k)
```

```
a = input()

while len(a) < 5:
  if a[:2].isdigit():
    continue

  print(a)
  a = input()
  if len(a) == 7:
    break

print(a)
```

continue: Используется для пропуска текущей итерации цикла и перехода к следующей

1.

```
for i in range(10):
      if i % 2 == 0:
          continue
      print(i) # вывод от 1 до 9 нечетных чисел
```

2.

```
count = 0
while count < 10:
      count += 1
      if count % 2 == 0:
          continue
      print(count)  # вывод от 1 до 9 нечетных чисел
```

```
# code
for i in range(10):
   if i % 2 == 0:
       continue
   print(i)
```

else после циклов

После цикла for или while можно использовать блок else, который будет выполнен только в том случае, если цикл закончился естественным образом, то есть без использования операторов break. Если в цикле был использован оператор break, то блок else будет проигнорирован.

1. Пример цикла for с else (блок else выполняется после завершения цикла for):

```
for num in range(5):
     print(num)
else:
     print("Числа закончились")
#Вывод
0
1
2
3
4
Числа закончились
```

2. Пример цикла for с else и break в цикле (из-за break блок else не выполняется):

```
for num in range(5):
     if num == 3:
         break
     else:
         print(num)
 else:
     print("Числа закончились")
#Вывод
0
1
2
```

continue не влияет на блок else

```
# code
for i in range(10): # [0, 8)
  if i % 2 == 0:
    print(i)
  if i == 15:
    break
else:
  print('Hello')
```

Вложенные циклы

Циклы могут быть вложенными друг в друга, что позволяет выполнять более сложные итерации. Если коротко, это цикл в цикле:

```
for i in range(1, 10):
    for j in range(1, 10):
        print(f"{i*j:4}", end=" ")
    print()
```

или

```
count1 = 0
while count1 <= 5:
  count2 = 0
  while count2 <=3:
    count2 += 1
  count1 += 1
```

Также можно:

```
for i in range(10):
  while ...
```

или наоборот

```
for i in range(1, 10, 2):
  print(i)
```

```

```

```

```

Функции

Определение и вызов функции

Мы изучили ряд встроенных в Python функций: **print()**, **input()**, **len()** и др.

Часто в программах требуется использовать какую-то часть кода несколько раз. В таком случае программисты создают функции. Функции помогают использовать написанный код в программах многократно, а также делают его более понятным за счёт деления на блоки.

```
def <имя функции>(<аргументы функции>):
    <тело функции>
```

Вызов функции:

```
<имя функции>(<аргументы функции>)
```

Пример:

$ax^2 + bx + c = 0$, найти x

$D = b^2 - 4ab > 0$

$$x_{1,2} = \frac{-b \pm \sqrt{D}}{2a}$$

Для решения 100 различных квадратных уравнения (разные a,b,c)требуется использовать какую-то эту часть кода несколько раз.

```
a = 1
b = 5
c = 1
D = b ** 2 - 4 * a * c
if D >= 0:
  x1 = (-b - D**0.5) / (2 * a)
  x2 = (-b + D**0.5) / (2 * a)
  print(x1, x2)
else:
  print('Нет реш.')
```

Эту проблему решают функции

```
def solver(a, b, c):
  D = b ** 2 - 4 * a * c
  if D >= 0:
    x1 = (-b - D**0.5) / (2 * a)
    x2 = (-b + D**0.5) / (2 * a)
    print(x1, x2)
  else:
    print('Нет реш.')
```

```
def solver(a, b, c):
  D = b ** 2 - 4 * a * c
  if D >= 0:
    x1 = (-b - D**0.5) / (2 * a)
    x2 = (-b + D**0.5) / (2 * a)
    print(x1, x2)
  else:
    print('Нет реш.')

solver(1,1,2)
solver(1,0,0)
solver(1,3,1)
```

Аргументы

Параметры — это переменные, которые принимают значения при вызове функции (аргументы).

Аргументы могут передаваться по позиции или по имени.

```
def add(a, b):
    return a + b

add(3, 5)  # 8
add(b=5, a=3)  # 8
```

```
# code
```

Можно задать значения по умолчанию для параметров, которые будут использоваться, если соответствующий аргумент не предоставлен при вызове функции.

```
def add(a, b=5):
    return a + b

add(3, 5)  # 8
add(3)  # 8
```

```
# code
```

Возвращаемое значение

Функция может возвращать значение с помощью оператора **return**. Если функция не возвращает значение явно, она возвращает **None** по умолчанию.

```
def add1(a, b):
    return a + b

result = add1(3, 5) # result = 8

def add2(a, b):
    a + b

result = add2(3, 5) # result = None
```

Переменные

Локальные и **глобальные** переменные:

Локальные. Переменные, определенные внутри функции, считаются локальными и существуют только в течение времени выполнения функции.

Глобальные. Переменные, определенные вне функций, считаются глобальными и доступны во всем коде.

```
pi = 3.14  # глобальная переменная

def test():
    pi = 5  # локальная переменная
    print(f"Local pi: {pi}") # 5

test()
print(f"Global pi: {pi}") # 3.14
```



Еще раз!

Переменные, созданные вне функций, то есть в основном коде программы, находятся в глобальной области видимости.

Пример 1

```
pi = 3.14  # глобальная переменная

def test():
    print(f"Local pi: {pi}") # 3.14

test()
print(f"Global pi: {pi}") # 3.14
```

Пример 2

```
l = [1, 23, 421] # глобальная переменная

def test():
    l.append(123)

test()
print(l) # [1, 23, 421, 123]
```

```
# code
```

Если в функции необходимо менять значения переменных из глобальной области видимости путём операции присваивания, то нужно в теле функции сделать эти переменные глобальными с помощью ключевого слова **global**. Тогда в функции не будут создаваться локальные переменные с такими же именами, а значение поменяется в глобальной переменной:

```
def inc():
    global x
    x += 1
    print(f"Количество вызовов функции равно {x}.")

x = 0
inc() #Количество вызовов функции равно 1
inc() # Количество вызовов функции равно 2.
inc() # Количество вызовов функции равно 3.
```

```
# code
```

Базовые функции

1. **input** - ввод
2. **print** - вывод
3. **max (min)** - максимум (минимум)

```
a,b, c = 0, 1, 51
max(a, b, c) # 51
l = [1, 2, 4]
max(l) # 4
l = ['a', 'r', 'я']
max(l) # я
```

4. **sorted(iterable[, key][, reverse])** - сортировка списка

```
l = [345, 234, 123]
sorted(l) # [123, 234, 345]
l = ['первый', 'второй', 'АПвторой']
sorted(l) # ['АПАПвторой', 'второй', 'первый'] # по алфавиту
l = ['первый', 'второй', '4ads','АПвторой']
sorted(l, key=len) # ['4ads', 'первый', 'второй', 'АПвторой'] # по длинне строки
l = ['первый', 'второй', '4ads','АПвторой']
sorted(l, key=len, reverse=True) # ['АПвторой', 'первый', 'второй', '4ads']  # в обратном порядке
см. пример выше
```

5. **abs** - модуль числа

```
abs(-23) # 23
```

6. **len** - кол-во элементов в указанном объекте

```
len([123, -123]) # 2
```

7. **reversed** - Итератор из развернутого объекта (перевернутый список элементов)

```
l = ['первый', 'второй', '4ads','АПвторой']
reversed(l) # <list_reverseiterator at 0x7a0225101630>
list(reversed(l)) # ['АПвторой', '4ads', 'второй', 'первый']
```

8. **round(X [, N])** - Округление до N знаков после запятой

```
pi = 3.14159265359
round(pi, 2) # 3.14
round(pi, 1) # 3.1
round(pi, 0) # 3.0
```

9. **sum(iter, start=0)** - сумма элементов

```
l = [12, 412, 142, 123]
sum(l) # 689
```

10. **range([start=0], stop, [step=1])** - арифметическая прогрессия от start до stop с шагом step.

```
range(0, 12, 2) # range(0, 12, 2)
list(range(0, 12, 2)) # [0, 2, 4, 6, 8, 10]
```

11. **map(function, iterator)** - Итератор, получившийся после применения к каждому элементу последовательности функции function. **map выполняет функцию для каждого элемента последовательности.**

```
l = ['12', '213', '31', '2'] # все значения строковые!
map(int, l) # <map at 0x7a0225103700>
```

```
list(map(int, l)) # [12, 213, 31, 2] # все значения int!
```

13. **enumerate** - Возвращает итератор, при каждом проходе предоставляющем кортеж из номера и соответствующего члена последовательности.

```
l = ['12', '213', '31', '2'] # все значения строковые!
enumerate(int, l) # <enumerate at 0x7a022502e9c0>
list(enumerate(int, l)) #[(0, '12'), (1, '213'), (2, '31'), (3, '2')]
```

```
# code
```

```

```

Доп *

1. **type(object)** - Возвращает тип объекта
2. **eval(expression, globals=None, locals=None)**

```
eval('1+2+3+4+5**2 + 1/4') # 35.25
```

3. **divmod(a, b)** - Возвращает частное и остаток от деления a на b.
4. **bin(x)** - Преобразование целого числа в двоичную строку.

Доп. функции https://pythonworld.ru/osnovy/vstroennye-funkcii.html (https://pythonworld.ru/osnovy/vstroennye-funkcii.html)

```
# code
```

```

```

Классы

В Python, класс (class) является основным механизмом для создания пользовательских типов данных и объектов. Класс определяет атрибуты (свойства) и методы (функции), которые могут быть использованы для работы с объектами этого класса. Ниже приведена основная информация о классах в Python.

Определение класса:

Класс определяется с помощью ключевого слова **class**, за которым следует имя класса и блок кода, заключенный в скобки. Например:

```
class Dog:
    pass
```

Атрибуты класса определяется внутри класса и могут быть использованы для хранения данных. Атрибуты класса определяются вне методов и доступны для всех объектов класса. Например:

```
class Dog:
    species = "Canis familiaris"

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

В этом примере определяется класс Dog с атрибутом класса species и двумя атрибутами объекта name и age, которые инициализируются в методе **init()**.

Методы класса:

Методы класса определяются внутри класса и могут быть использованы для выполнения различных действий с объектами этого класса. Методы определяются с помощью ключевого слова def и принимают первый параметр self, который ссылается на текущий объект. Например:

```
class Dog:
    species = "Canis familiaris"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        return f"{self.name} says: Woof!"

    def birthday(self):
        self.age += 1
```

В этом примере определяется класс Dog с двумя методами bark() и birthday(). Метод bark() возвращает строку с именем собаки и звуком лая, а метод birthday() увеличивает возраст собаки на 1.

```
class Dog:
    species = "Canis familiaris"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self, a):
        return f"{self.name} {a} says: {'Woof'*a}!"

    def birthday(self):
        self.age += 1
```

```
class Cat:
    species = "Canis familiaris"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def mew(self, a):
        return f"{self.name} {a} says: {'mew'*a}!"

    def birthday(self):
        self.age += 1
```

```
dogs = Dog('Кот', 50)
cats = Cat('Dog', 50)
```

```
cats.mew(1)
```

```
dogs.species
```

```
dogs.bark(2)
```

```
# [1, 2, 3, 4] + 10
# [10, 12, 13, 14]
```

```
class vector(list):
  def __add__(self, int):
    for i in range(len(self)):
      self[i] += int
    return self
```

```
vector([1, 2, 3, 4]) + 20
```

```
new_s = new_str('FBB')
new_s + 10
```

Создание объекта:

Объекты класса создаются с помощью оператора `<class_name>()` и могут быть использованы для доступа к атрибутам и методам класса. Например:

```
my_dog = Dog("Buddy", 3)
print(my_dog.name)  # Buddy
print(my_dog.bark())  # Buddy says: Woof!
my_dog.birthday()
print(my_dog.age)  # 4
```

В этом примере создается объект my_dog класса Dog с именем "Buddy" и возрастом 3. Затем доступ к атрибутам и методам объекта осуществляется с помощью точечной нотации (.).

```

```

Можно переопределять (наследовать и расширять) встроенные типы, такие как list, dict, str, int, set и т.д. Это позволяет создавать собственные версии стандартных типов с дополнительной функциональностью или изменённым поведением.

Перегрузка операторов: https://pythonworld.ru/osnovy/peregruzka-operatorov.html (https://pythonworld.ru/osnovy/peregruzka-operatorov.html)

```
class UpperStr(str):
    def __add__(cls, int):
        return f'{cls}{int}'

# Использование
s = UpperStr("hello")
```

```

```

Чтение файла в питоне

В Python, чтение файла можно осуществить с помощью функции **open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)** или **open()**, которая открывает файл и возвращает объект файла. В этом объекте можно использовать метод **read()** для чтения всего содержимого файла или метод **readline()** для чтения одной строки за раз.

Документация (https://docs.python.org/3/library/functions.html#open)

Например, для чтения всего содержимого файла "example.txt" можно использовать следующий код:

```
file = open("example.txt", "r")
content = file.read()
print(content)
file.close()
```

В этом примере "r" указывает на то, что файл открывается для чтения.

```
Character

Meaning

'r' – чтение (read) : Открывает файл для чтения. Если файл не существует, то попытка открыть его приведет к ошибке FileNotFoundError. Если файл уже открыт в режиме записи ("w" или "a"), то попытка открыть его в режиме чтения ("r") приведет к ошибке FileExistsError.

'w' – запись (write) : Открывает файл для записи. Если файл уже существует, то он будет перезаписан. Если файла не существует, то он будет создан. Если файл уже открыт в режиме чтения ("r"), то попытка открыть его в режиме записи ("w") приведет к ошибке FileExistsError.

'a' – добавление (append) : Открывает файл для добавления информации в конец файла. Если файла не существует, то он будет создан. Если файл уже открыт в режиме чтения ("r"), то попытка открыть его в режиме добавления ("a") приведет к ошибке FileExistsError.

't' – text mode (default) – Режим текстового режима означает, что файл будет открыт в режиме чтения или записи с автоматическим преобразованием символов новой строки в символы перевода строки, соответствующие операционной системе.

'+' – open for updating (reading and writing)
```

Также можно использовать конструкцию **with**, которая автоматически закрывает файл после завершения работы с ним:

```
with open("example.txt", "r") as file:
    content = file.read()
print(content)
```

```
# code
# file = open("example.txt", "r")
with open("example.txt", "r") as file:
  content = file.read()
print(content)
```

Также можно читать файл по строкам, используя метод readline():

```
with open("example.txt", "r") as file:
    for line in file:
        print(line)
```

В этом примере каждая строка файла будет распечатываться отдельно.

Также можно использовать методы readlines() и iterlines() для чтения файла по строкам:

```
with open("example.txt", "r") as file:
    lines = file.readlines()
```

```
for line in lines:
    print(line)

with open("example.txt", "r") as file:
    for line in file.iterlines():
        print(line)
```

```
# code
with open("example_write.txt", "w") as file:
  file.write('123')
```